

Al-Hamzeh LMS

What exists in the code today, how the pieces fit together, and where the sharp edges are. Written for engineers extending, operating, or auditing the application.

REPOSITORY LMS-Al-Hamzeh	PACKAGE dweiri-lms
PRODUCT NAME GATA3A Academy	BRANCH chore/production-hardening

Issued 15 September 2026

CONTENTS

OVERVIEW

01	What the app does	2
02	Technology stack	2
03	Repository layout	3
04	Environment variables	4

DATA & IDENTITY

05	Data model	4
06	Authentication	6
07	Authorization model	6

APPLICATION

08	Routing map	8
09	Data access layer	9
10	Server actions	11
11	HTTP API routes	12

SUBSYSTEMS

12	File storage and uploads	13
13	Payments and enrollment	14
14	Progress tracking	15
15	Course materials	15
16	Content protection	16
17	Abuse protection	17

FRONTEND

18	UI system	18
19	Shared hooks and helpers	18
20	Validation schemas	19

OPERATIONS

21	Local development	20
22	Build and deployment	20
23	Maintenance scripts	21

REFERENCE

24	Known issues and technical debt	22
25	Conventions	24

01 What the app does

A paid, multi-tenant-ish learning management system for Jordanian universities, serving three audiences.

AUDIENCE	ENTRY POINT	CAPABILITIES
Visitor	<code>/</code> <code>/courses</code>	Browse published courses, view course detail pages, sign in
Student	<code>/</code> <code>dashboard</code>	See enrolled courses, watch video lessons, read PDF materials, track progress
Admin	<code>/admin</code>	Full CRUD on courses, chapters and lessons; upload materials; manage users, instructors and students; view analytics

Courses are sold individually through Stripe Checkout. Access to lesson content and materials is gated on an `Enrollment` row with status `Active`, which the Stripe webhook sets after a successful payment.

Content — course thumbnails, lesson videos, material PDFs — lives in a Tigris S3-compatible bucket (`*.fly.storage.tigris.dev`). Videos and images are served from the bucket host directly; material PDFs are served through short-lived presigned URLs generated by the app.

02 Technology stack

CONCERN	CHOICE	NOTES
Framework	Next.js 15.3 (App Router)	React 19, React Server Components, <code>--turbo</code> in dev
Language	TypeScript 5	Strict mode, path alias <code>@/*</code> → repo root
Database	PostgreSQL	Accessed only through Prisma
ORM	Prisma 6.8	<code>prisma generate</code> runs on <code>postinstall</code>
Auth	better-auth 1.2	Prisma adapter, <code>emailOTP</code> + <code>admin</code> plugins, GitHub OAuth
Payments	Stripe 18	API <code>2025-08-27.basil</code> ; Checkout Sessions + webhook
Storage	AWS SDK v3 S3 → Tigris	Presigned PUT for media, presigned GET for materials
Email	Resend	Only used to deliver sign-in OTPs
Abuse protection	Arcjet (<code>@arcjet/next</code>)	Shield baseline, per-route rate limits, bot and signup protection
Styling	Tailwind CSS v4	+ <code>tw-animate-css</code> , <code>@tailwindcss/typography</code> ; CSS-first config in <code>app/globals.css</code>
Components	shadcn/ui (Radix)	<code>components.json</code> , “new-york” style
Forms	react-hook-form + Zod	via <code>@hookform/resolvers</code>

CONCERN	CHOICE	NOTES
Rich text	TipTap 2	<code>starter-kit</code> , <code>text-align</code> ; stored as HTML, rendered with <code>html-react-parser</code>
Drag and drop	<code>@dnd-kit</code>	Chapter and lesson reordering
Charts	Recharts	Admin enrollment chart
PDF rendering	<code>pdfjs-dist</code> 5	Canvas-based custom viewer, worker at <code>/pdf.worker.min.js</code>
Toasts	Sonner	Mounted once in the root layout
Theme	<code>next-themes</code>	Class-based dark mode, <code>defaultTheme="system"</code>
Icons	lucide-react, Tabler	Both are in use

03 Repository layout

```

app/
  layout.tsx      Root layout: Geist fonts, ThemeProvider, Toaster
  globals.css     Tailwind v4 entry + oklch design tokens (light/dark)
  (public)/       Marketing + catalog, wrapped in Navbar layout
  (auth)/         Login and OTP verification, centered card layout
  admin/         Admin console, guarded by requireAdmin in its layout
  dashboard/     Student area, guarded per-page by requireUser
  payment/       Stripe return pages (success, cancel)
  not-admin/     Rejection page for non-admins
  api/           Route handlers – see §11
  data/          Server-only data access + guards – see §9
components/
  ui/            shadcn/ui primitives, SecurePDFViewer, themeToggle
  sidebar/       Admin shell: app-sidebar, site-header, nav-*, cards, chart
  file-uploader/ Uploader (dropzone + presigned PUT), RenderState
  rich-text-editor/ TipTap Editor, Menubar, RenderDescription
  general/EmptyState.tsx
hooks/
  useCourseProgress, useConfetti, useSignOut, useIsMobile,
  useConstructUrl, tryCatch
lib/
  auth, auth-client, db, env, S3Client, stripe, arcjet, resend,
  api-auth, zodSchemas, types, utils
prisma/schema.prisma Single source of truth for the DB
public/
  logo.png, logo1.png, pdf.worker.min.js
scripts/
  One-off orphan cleanup scripts (run with tsx)
next.config.ts   Image remote patterns

```

CONVENTION Route-local components live in a `_components/` folder next to the route that uses them; route-local server actions live in an `actions.ts` beside the route. Shared components go in `components/`.

04 Environment variables

Validated at import time by `lib/env.ts` using `@t3-oss/env-nextjs` and Zod. A missing or malformed variable fails the build or boot rather than failing at runtime.

SERVER-ONLY

VARIABLE	PURPOSE
<code>DATABASE_URL</code>	Postgres connection string (must be a URL)
<code>BETTER_AUTH_SECRET</code>	Session signing secret
<code>BETTER_AUTH_URL</code>	Public base URL; also builds Stripe <code>success_url</code> / <code>cancel_url</code>
<code>AUTH_GITHUB_ID</code> <code>AUTH_GITHUB_SECRET</code>	GitHub OAuth app credentials
<code>RESEND_API_KEY</code>	Sends sign-in OTP emails
<code>ARCJET_KEY</code>	Arcjet site key
<code>AWS_ACCESS_KEY_ID</code> <code>AWS_SECRET_ACCESS_KEY</code>	Tigris credentials
<code>AWS_ENDPOINT_URL_S3</code> <code>AWS_ENDPOINT_URL_IAM</code> <code>AWS_REGION</code>	Tigris endpoints and region
<code>STRIPE_SECRET_KEY</code>	Stripe API key
<code>STRIPE_WEBHOOK_SECRET</code>	Verifies webhook signatures

CLIENT-EXPOSED

VARIABLE	PURPOSE
<code>NEXT_PUBLIC_S3_BUCKET_NAME_IMAGES</code>	Bucket name; used both as the S3 <code>Bucket</code> and to build public URLs

NOTE The single bucket variable is used for *all* object types — images, videos and materials — despite its name.

DATA & IDENTITY

05 Data model

Defined in `prisma/schema.prisma`, provider `postgresql`. Better-auth tables are mapped to lowercase names (`@@map("user")`, `"session"`, `"account"`, `"verification"`, `"material"`); course-domain tables use Prisma's default names.

Relationships at a glance

```
graph TD
    User -->|creator · userId| Course1[Course]
    User -->|instructor · instructorId (optional)| Course2[Course]
    User --> Enrollment --> Course3[Course]
    User --> LessonProgress --> Lesson
    Course --> Chapter --> Lesson
    Course --> Material
```

Auth tables — owned by better-auth

MODEL	FIELDS AND BEHAVIOUR
User	<code>id</code> (string, supplied by better-auth), <code>name</code> , <code>email</code> (unique), <code>emailVerified</code> , <code>image</code> , timestamps. Extended with <code>stripeCustomerId</code> (unique, nullable, set lazily on first checkout); <code>role</code> (nullable string: "admin", "instructor", or student); <code>banned</code> , <code>banReason</code> , <code>banExpires</code> from the admin plugin. Relations: <code>sessions</code> , <code>accounts</code> , <code>courses</code> , <code>instructorCourses</code> , <code>enrollment</code> , <code>lessonProgress</code> .
Session	Token, expiry, IP, user agent, <code>impersonatedBy</code> . Cascades on user delete.
Account	OAuth / credential records per provider. Cascades on user delete.
Verification	OTP and verification tokens.

Course domain

MODEL	FIELDS AND BEHAVIOUR
Course	<code>id</code> (uuid), <code>title</code> , <code>description</code> (TipTap HTML), <code>fileKey</code> (thumbnail), <code>price</code> (Int, whole currency units), <code>duration</code> (Int hours), <code>level</code> , <code>stripePriceId</code> unique , <code>category</code> , <code>smallDescription</code> , <code>slug</code> unique , <code>status</code> , <code>university</code> (default UJ), timestamps. <code>userId</code> → creator, cascade delete. <code>instructorId</code> → optional instructor, no cascade. Children: <code>chapter[]</code> , <code>enrollment[]</code> , <code>materials[]</code> .
Chapter	<code>title</code> , <code>position</code> (Int, 1-based), <code>courseId</code> (cascade), <code>lessons[]</code>
Lesson	<code>title</code> , <code>description</code> (optional HTML), <code>thumbnailKey</code> , <code>videoKey</code> , <code>position</code> , <code>chapterId</code> (cascade), <code>lessonProgress[]</code>
Enrollment	<code>amount</code> (Int), <code>status</code> (default Pending), <code>userId</code> + <code>courseId</code> (both cascade), <code>@@unique([userId, courseId])</code>
LessonProgress	<code>completed</code> (Boolean, default false), <code>userId</code> + <code>lessonId</code> (both cascade), <code>@@unique([userId, lessonId])</code>
Material	<code>title</code> , <code>fileKey</code> , <code>isVisible</code> (default true), <code>courseId</code> (cascade), timestamps

Enums

University	UJ PETRA
CourseLevel	Beginner Intermediate Advanced
CourseStatus	Draft Published Archived
EnrollmentStatus	Pending Active Cancelled

Migration strategy

RISK There is **no** `prisma/migrations/` **directory**. The schema is applied with `prisma db push`, so there is no migration history and no reproducible way to replay schema changes. Introducing proper migrations is the single highest-value schema change before further production work.

06 Authentication

`lib/auth.ts` configures better-auth:

- **Adapter** — `prismaAdapter(prisma, { provider: "postgresql" })`
- **Social provider** — GitHub, via `AUTH_GITHUB_ID` / `AUTH_GITHUB_SECRET`
- **emailOTP plugin** — sends a one-time code via Resend from `onboarding@resend.dev`. Outside production the OTP is also logged to the console and a delivery failure is swallowed with a warning, so local sign-in works without a verified domain. In production a delivery failure throws.
- **admin() plugin** — supplies `role`, `banned`, `banReason`, `banExpires`, and session impersonation.

`lib/auth-client.ts` is the browser counterpart: `createAuthClient` with `emailOTPClient()` and `adminClient()`. Client components read the session reactively with `authClient.useSession()`.

Sign-in flow

- 1 `/login` renders `LoginForm`, offering “Sign in with GitHub” or email.
- 2 Email path calls `authClient.emailotp.sendVerificationOtp({ email, type: "sign-in" })`, then redirects to `/verify-request?email=...`.
- 3 `/verify-request` collects the 6-digit code with `input-otp` and verifies it.
- 4 Sign-out is centralized in `useSignOut` (`hooks/use-signout.ts`), which pushes `/` and toasts.

All better-auth HTTP traffic goes through `app/api/auth/[...all]/route.ts`, which wraps the handler in Arcjet protections (§17).

07 Authorization model

There is **no** `middleware.ts`. Every protected surface guards itself, through three deliberately redundant layers.

LAYER 1 PAGES	Page and layout guards · <code>app/data/*/require-*.ts</code> Wrapped in React's <code>cache()</code> so a request pays for at most one session lookup. <code>requireUser()</code> returns <code>session.user</code> or redirects to <code>/login</code>. <code>requireAdmin()</code> returns the session, redirecting to <code>/login</code> when unauthenticated and <code>/not-admin</code> when <code>role !== "admin"</code>. <code>app/admin/layout.tsx</code> calls <code>requireAdmin()</code>, covering every current and future page under <code>/admin</code>.
LAYER 2 DATA	Data-function guards Every function in <code>app/data/admin/</code> begins with <code>await requireAdmin()</code>; every student-scoped function in <code>app/data/user/</code> and <code>app/data/course/</code> (except public catalog readers) begins with <code>await requireUser()</code>. A page that bypassed its layout still cannot read admin data. <code>getCourseSidebarData</code> and <code>getLessonContent</code> go further and call <code>notFound()</code> unless an <code>Active</code> enrollment exists.
LAYER 3 ROUTES	Route-handler guards · <code>lib/api-auth.ts</code> Redirect-based guards are wrong for <code>fetch</code> callers. These helpers resolve to <code>{ session }</code> or <code>{ error: NextResponse }</code> with JSON, and are called <i>before</i> <code>try/catch</code> so an auth failure can't be swallowed into a 500.

HELPER	RULE	FAILURE
<code>requireApiUser()</code>	Any signed-in user	401 Authentication required
<code>requireApiAdmin()</code>	<code>role === "admin"</code>	403 Admin access required
<code>requireCourseAccess(courseId)</code>	Admin, or an <code>Active</code> enrollment in that course	403 Course enrollment required

Roles

Roles are free-form strings on `User.role`:

- `"admin"` — full access to `/admin`
- `"instructor"` — appears in instructor pickers and the instructors console; **grants no elevated access by itself**
- `null` or anything else — treated as a student

Students are identified implicitly: admin “students” listings query users with at least one enrollment.

08 Routing map

Public • `app/(public)/`

The layout wraps children in `Navbar` and a centered container.

ROUTE	FILE	NOTES
<code>/</code>	<code>page.tsx</code>	Static marketing hero and four feature cards
<code>/courses</code>	<code>courses/page.tsx</code>	<code>force-dynamic</code> ; <code>Suspense</code> + skeleton grid, renders <code>CoursesList</code>
<code>/courses/[slug]</code>	<code>courses/[slug]/page.tsx</code>	Thumbnail, description, collapsible chapters, badges, purchase / continue CTA, materials section

`/courses` filtering is client-side: the server sends all published courses and `CoursesFilters` narrows by search, instructor, university, category and level. The admin equivalent filters by search, instructor, university and status.

Auth • `app/(auth)/`

ROUTE	NOTES
<code>/login</code>	<code>LoginForm</code> : GitHub button and email OTP request
<code>/verify-request</code>	OTP entry, reads <code>?email=</code>

Student • `app/dashboard/`

Shell layout: `SidebarProvider` + `DashboardAppSidebar` + `SiteHeader` .

ROUTE	NOTES
<code>/dashboard</code>	Enrolled courses as <code>CourseProgressCard</code> s, plus unpurchased courses as <code>PublicCourseCard</code> s
<code>/dashboard/[slug]</code>	Guarded by <code>getCourseSidebarData</code> ; redirects to the first lesson or shows “No lessons available”
<code>/dashboard/[slug]/[lessonId]</code>	<code>Suspense</code> + <code>LessonSkeleton</code> , then <code>CourseContent</code> (player, description, mark-complete)
<code>/dashboard/[slug]/materials</code>	<code>DashboardMaterials</code> — lists visible materials and opens them in the secure viewer

`app/dashboard/[slug]/layout.tsx` fetches sidebar data once and renders `MobileCourseSidebar` below `lg` , `CourseSidebar` at `lg` and above.

Admin • `app/admin/`

ROUTE	NOTES
<code>/admin</code>	<code>SectionCards</code> (4 KPIs), <code>ChartAreaInteractive</code> (30-day enrollments), two most recent courses
<code>/admin/courses</code>	Filterable grid (<code>CoursesList</code> + <code>AdminCourseCard</code>)
<code>/admin/courses/create</code>	<code>CourseCreationForm</code> — creates the Stripe product and the course
<code>/admin/courses/[courseId]/edit</code>	Tabs: Basic Info (<code>EditCourseForm</code>) and Course Structure (<code>CourseStructure</code>)
<code>/admin/courses/[courseId]/[chapterId]/[lessonId]</code>	<code>LessonForm</code> — title, description, thumbnail, video
<code>/admin/courses/[courseId]/delete</code>	Confirmation page for <code>deleteCourse</code>
<code>/admin/materials</code>	<code>MaterialsHeader</code> (upload dialog) + <code>MaterialsList</code>
<code>/admin/materials/[id]</code>	<code>MaterialDetails</code> — metadata and inline secure preview
<code>/admin/students</code>	<code>StudentsStatsCards</code> + <code>StudentsTable</code>
<code>/admin/students/[studentId]</code>	<code>StudentDetailsView</code> — per-course progress breakdown
<code>/admin/instructors</code>	<code>InstructorsTable</code> with create, edit, delete
<code>/admin/universities</code>	<code>UniversitiesStatsCards</code> — per-university counts
<code>/admin/user-management</code>	<code>UserManagementTable</code> — ban / unban, change role

Sidebar navigation is a literal array in `components/sidebar/app-sidebar.tsx`. `NavMain` highlights the active item via `usePathname()` and shows “Quick Create” only under `/admin`.

Other

ROUTE	NOTES
<code>/not-admin</code>	Shown when a signed-in non-admin hits <code>/admin</code>
<code>/payment/success</code>	Stripe <code>success_url</code> ; fires confetti
<code>/payment/cancel</code>	Stripe <code>cancel_url</code>

09 Data access layer

Every file in `app/data` starts with `import "server-only"`, so importing one into a client component is a build error rather than a leak. Modules export a query function and, where components consume the shape, an inferred type — `AdminCourseType`, `PublicCourseType`, `LessonContentType`, `CourseSidebarDataType`, `EnrolledCourseType`, `AdminCourseSingularType`, `AdminLessonType`. Widening a `select` automatically widens the component contract.

app/data/admin/

FUNCTION	RETURNS
<code>requireAdmin()</code>	Cached admin session guard
<code>adminGetCourses()</code>	All courses, newest first, with instructor summary
<code>adminGetRecentCourses()</code>	Same shape, <code>take: 2</code>
<code>adminGetCourse(id)</code>	One course with chapters and lessons; <code>notFound()</code> when missing
<code>adminGetLesson(id)</code>	One lesson; <code>notFound()</code> when missing
<code>adminGetDashboardStats()</code>	<code>{ totalSignups, totalCustomers, totalCourses, totalLessons }</code> via one <code>Promise.all</code>
<code>adminGetEnrollmentStats()</code>	30 zero-filled <code>{ date, enrollments }</code> buckets
<code>getMaterials()</code> , <code>getMaterialById(id)</code> , <code>getMaterialsByCourseId(id)</code> , <code>getCourses()</code>	Materials console reads; each guards with <code>requireAdmin()</code> and returns <code>[]</code> / <code>null</code> on error

app/data/user/

FUNCTION	NOTES
<code>requireUser()</code>	Cached user guard
<code>getEnrolledCourses()</code>	<code>Active</code> enrollments with nested chapters, lessons and the caller's <code>lessonProgress</code> — enough to compute progress client-side
<code>checkIfCourseBought(courseId)</code>	<code>boolean</code> ; returns <code>false</code> for anonymous visitors instead of redirecting

app/data/course/

FUNCTION	GUARD
<code>getAllCourses()</code>	None — public, <code>status: "Published"</code> only
<code>getIndividualCourse(slug)</code>	None — public detail page; chapters and lesson titles, no <code>videoKey</code>
<code>getCourseSidebarData(slug)</code>	<code>requireUser()</code> + <code>Active</code> enrollment, else <code>notFound()</code>
<code>getLessonContent(lessonId)</code>	<code>requireUser()</code> + <code>Active</code> enrollment on the lesson's course, else <code>notFound()</code>

BY DESIGN `getIndividualCourse` deliberately omits `videoKey` and `thumbnailKey`, so the public detail page cannot leak playable media URLs.

10 Server actions

All are "use server" files colocated with their route, returning the shared `ApiResponse = { status: "success" | "error"; message: string }` from `lib/types.ts` (some widen it with `data` or `lessonId`). Callers wrap them in `tryCatch` and surface `message` through Sonner.

app/admin/courses/create/actions.ts

- `getInstructors()` — users with `role: "instructor"`, for the instructor picker.
- `CreateCourse(values)` — `requireAdmin()` → Arcjet fixed window (5/min) → `courseSchema.safeParse` → `stripe.products.create` with `default_price_data` (`price * 100`, USD) → `prisma.course.create` storing `stripePriceId` and `userId`.

app/admin/courses/[courseId]/edit/actions.ts

- `editCourse(data, courseId)` — `requireAdmin()` → Arcjet → validate → update. The **where** clause includes `userId: user.userId`, so an admin can only edit courses they created.
- `reorderChapters / reorderLessons` — position updates inside a `$transaction`, then `revalidatePath`.
- `createChapter / createLesson` — compute the next `position` from the current max in a transaction. `createLesson` returns the new `lessonId` so the UI can navigate to it.
- `deleteChapter / deleteLesson` — renumber surviving siblings to `1..n` and delete, in one transaction.

app/admin/courses/[courseId]/delete/actions.ts

- `deleteCourse(courseId)` — `requireAdmin()` → Arcjet → `prisma.course.delete` (chapters, lessons, enrollments and materials cascade) → `revalidatePath("/admin/courses")`.

app/admin/courses/[courseId]/[chapterId]/[lessonId]/actions.ts

- `updateLesson(values, lessonId)` — validates with `lessonSchema`; updates title, description, `thumbnailKey`, `videoKey`.

app/admin/user-management/actions.ts

- `banUserAction(id, reason, expires?)`, `unbanUserAction(id)`, `updateUserRoleAction(id, role)` — direct Prisma writes to the better-auth fields, then `revalidatePath`.

app/admin/instructors/actions.ts

- `getInstructors()` — summaries with course counts. An instructor's `university` is **derived from their first course**, defaulting to "UJ".
- `getInstructorDetails(id)` — adds per-course enrollment counts.
- `createInstructor(name, email)` — rejects duplicate emails, then inserts a `User` with `id: crypto.randomUUID()`, `role: "instructor"`, `emailVerified: false`. No `Account` row; the person signs in through email OTP.
- `updateInstructor(id, name, email)` — rejects an email taken by another user.
- `deleteInstructor(id)` — refuses while the instructor still has assigned courses.

- `getUniversityStats()` — course, instructor and student counts for `UJ` and `PETRA`, hardcoded as two branches.

app/admin/students/actions.ts

- `getStudentsStats()` — totals plus an average completion rate computed in application code by loading every student's enrollments, chapters, lessons and progress rows.
- `getStudents()` / `getStudentDetails(id)` — same style, per student.

app/(public)/courses/[slug]/actions.ts

- `enrollInCourseAction(courseId)` — the checkout entry point (§13).

app/dashboard/[slug]/[lessonId]/actions.ts

- `markLessonComplete(lessonId, slug)` — `requireUser()` → `lessonProgress.upsert` on `userId_lessonId` → `revalidatePath('/dashboard/{slug}')`.

11 HTTP API routes

ROUTE	METHODS	AUTH	PURPOSE
<code>/api/auth/[...all]</code>	GET, POST	n/a	better-auth handler; <code>POST</code> wrapped in Arcjet
<code>/api/webhook/stripe</code>	POST	Stripe signature	Activates enrollments on <code>checkout.session.completed</code>
<code>/api/s3/upload</code>	POST	<code>requireAdmin()</code> + Arcjet 5/min	Presigned <code>PutObject</code> URL (6-min expiry) and object key
<code>/api/s3/delete</code>	DELETE	<code>requireAdmin()</code> + Arcjet 5/min	Deletes an object by key
<code>/api/materials/upload</code>	POST	<code>requireApiAdmin()</code>	Multipart upload to <code>materials/{courseId}/{timestamp}-{name}</code> ; creates the <code>Material</code> row
<code>/api/materials/[id]</code>	GET, DELETE	GET: <code>requireCourseAccess</code> DELETE: <code>requireApiAdmin</code>	Material metadata; delete S3 object then row
<code>/api/materials/[id]/visibility</code>	PATCH	<code>requireApiAdmin()</code>	Toggles <code>isVisible</code>
<code>/api/materials/[id]/view</code>	GET	<code>requireCourseAccess()</code>	Returns <code>{ url }</code> — 1-hour presigned GET, or the stored absolute URL
<code>/api/courses/[courseId]/materials</code>	GET	Inline session + <code>Active</code> enrollment	Visible materials for an enrolled student (<code>id</code> , <code>title</code> , <code>createdAt</code>)

Materials routes

These carry most of the access-control weight.

- `/view` looks up the material *first* to resolve `courseId`, then calls `requireCourseAccess(material.courseId)`. Without that gate, anyone holding a material id could download paid content. It also 404s a hidden material for non-admins.
- `/view` responses carry `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin` and no-store cache headers. The URL is returned as JSON rather than a redirect, because Chrome blocks cross-origin redirects inside the sandboxed iframe the viewer uses.
- `fileKey` may hold an S3 key or an absolute `http(s)` URL (seeded rows). Both `/view` and `useConstructUrl` detect the latter with `/^https?:\\\/\\\/i` and pass it through.
- The old unauthenticated `app/api/materials/route.ts` (public list and upload) has been **deleted** on this branch. Nothing in the UI referenced it.

Client call sites

<code>components/file-uploader/Uploader.tsx</code>	→ <code>POST /api/s3/upload</code> , <code>DELETE /api/s3/delete</code>
<code>admin/materials/.../MaterialsHeader.tsx</code>	→ <code>POST /api/materials/upload</code>
<code>admin/materials/.../MaterialCard.tsx</code>	→ <code>PATCH /api/materials/{id}/visibility</code> <code>DELETE /api/materials/{id}</code>
<code>admin/materials/[id]/.../MaterialDetails</code>	→ <code>GET /api/materials/{id}</code> <code>GET /api/materials/{id}/view</code>
<code>dashboard/[slug]/.../DashboardMaterials</code>	→ <code>GET /api/courses/{courseId}/materials</code> <code>GET /api/materials/{id}/view</code>
<code>(public)/courses/[slug]/.../SecurePDFViewer</code>	→ <code>GET /api/materials/{id}/view</code>

SUBSYSTEMS

12 File storage and uploads

`lib/S3Client.ts` creates one server-only `S3Client` with `region: "auto"`, the Tigris endpoint and `forcePathStyle: false`. There are two distinct upload paths.

A · Thumbnails and videos — presigned PUT, browser → S3

Implemented in `components/file-uploader/Uploader.tsx`.

- 1 `react-dropzone` accepts one image or video.
- 2 `POST /api/s3/upload` with `{ fileName, contentType, size, isImage }`. The route validates with Zod, rate-limits on the admin's user id, generates `key = ${uuidv4()}-${fileName}`, and returns `{ presignedUrl, key }` with a 360-second expiry.
- 3 The browser `PUT`s the file with `XMLHttpRequest` to report progress.
- 4 The `key` is written into the form field — `fileKey`, `thumbnailKey` or `videoKey`.
- 5 Removing a file calls `DELETE /api/s3/delete` with `{ key }`.

Bytes never pass through the Next.js server, so this path is not subject to serverless body-size limits.

B · Course materials — server-side upload, browser → Next.js → S3

`POST /api/materials/upload` receives `multipart/form-data`, buffers the whole file with `Buffer.from(await file.arrayBuffer())`, uploads it under `materials/{courseId}/{Date.now()}-{name}`, and creates the `Material` row.

LIMIT The entire file is held in server memory and bounded by the platform request-body limit — roughly 4.5 MB on Vercel serverless functions. Large PDFs will fail.

Reading objects back

- **Images and videos** — `useConstructUrl` builds `https://{BUCKET}.fly.storage.tigris.dev/{key}`: a public URL, no signature.
- **Materials** — always through `/api/materials/[id]/view`, which returns a 1-hour presigned URL.
- **next.config.ts** allow-lists `lms-alhamzeh.fly.storage.tigris.dev` for `next/image`, and adds `picsum.photos`, `fastly.picsum.photos` and `images.unsplash.com` only outside production.

13 Payments and enrollment

Product creation

Creating a course creates a Stripe **product** with inline `default_price_data` in USD at `price * 100` cents. The price id is stored on `Course.stripePriceId`.

DRIFT Editing a course's `price` updates only the database row. **The Stripe price is not updated**, so checkout keeps charging the original amount.

Checkout · `enrollInCourseAction(courseId)`

- 1 `requireUser()`.
- 2 Arcjet fixed window, 5/min, keyed on the user id.
- 3 Load the course: `id`, `title`, `price`, `slug`, `stripePriceId`.
- 4 Resolve the Stripe customer — reuse `user.stripeCustomerId`, or `stripe.customers.create` with `metadata.userId` and persist it.
- 5 In a `prisma.$transaction`: return early if an `Active` enrollment exists; otherwise create a `Pending` enrollment or reset a non-active one to `Pending`; then create a Checkout Session (`mode: "payment"`, one line item, return URLs from `BETTER_AUTH_URL`) with `metadata: { userId, courseId, enrollmentId }`.
- 6 `redirect(checkoutUrl)` — deliberately outside `try/catch`, since Next.js implements `redirect()` by throwing.

`Stripe.errors.StripeError` maps to “Payment system error”; anything else to “Failed to enroll in course”.

Webhook · `app/api/webhook/stripe/route.ts`

- 1 Read the raw body and verify `Stripe-Signature` against `STRIPE_WEBHOOK_SECRET`; a bad signature returns `400`.
- 2 On `checkout.session.completed`, read `metadata.courseId` and the `customer` id, find the user by `stripeCustomerId`, and set the enrollment identified by `metadata.enrollmentId` to `Active` with `amount: session.amount_total`.
- 3 Return `200` with an empty body.

A missing `courseId` or unknown customer throws, which surfaces as a 500 and causes Stripe to retry.

Gating

Once `Enrollment.status === "Active"`:

- `checkIfCourseBought` flips the detail CTA from “Enroll” to “Watch Course”.
- `getCourseSidebarData` and `getLessonContent` stop calling `notFound()`.
- `requireCourseAccess` starts allowing material views.

14 Progress tracking

- Completion is stored per `(userId, lessonId)` in `LessonProgress`, upserted by `markLessonComplete`.
- `useCourseProgress` derives `{ totalLessons, completedLessons, progressPercentage }` from the nested course tree in a `useMemo`. Because the readers already scope `lessonProgress` to the current user, completion is `lesson.lessonProgress.some(p => p.lessonId === lesson.id && p.completed)`.
- The same hook feeds `CourseSidebar`, `MobileCourseSidebar` and `CourseProgressCard` — the number a student sees is computed in exactly one place.
- Completing a lesson fires `useConfetti()`, a five-burst `canvas-confetti` preset.
- Admin-side progress is recomputed independently in `app/admin/students/actions.ts` by looping in application code.

15 Course materials

Materials are per-course files — in practice PDFs — uploaded by admins, stored in S3, and exposed to enrolled students through a protected viewer. There is no `Material` -to- `Lesson` relationship; materials hang off the course.

ADMIN

`/admin/materials` lists every material with its course. `MaterialsHeader` holds the upload dialog — title, course `Select` from `getCourses()`, file input — posting to `/api/materials/upload` then `router.refresh()`.

`MaterialCard` toggles visibility and deletes. `/admin/materials/[id]` fetches metadata, then `/view`, and previews inline.

STUDENT `/dashboard/[slug]/materials` lists visible materials via `/api/courses/{courseId}/materials` — a response that intentionally contains no `fileKey`.

Opening one calls `/api/materials/{id}/view` and renders in `SecurePDFViewer`. `CourseMaterialsSection` on the public page shows a teaser.

Deletion semantics. `DELETE /api/materials/[id]` attempts the S3 delete first, logs but ignores a failure, then deletes the row — a storage error cannot leave an undeletable record.

16 Content protection

SCOPE The app layers several deterrents around paid media. They raise the effort bar; none is a cryptographic guarantee, and that should be stated plainly to stakeholders.

Video • `CourseContent.tsx`

The `VideoPlayer`, defined inside `CourseContent`:

- renders a native `<video>` with `controls`, `controlsList="nodownload"`, `preload="metadata"`, `playsInline` and a poster from `thumbnailKey`;
- blocks `F12`, `Ctrl+Shift+I/J/C`, `Ctrl+U` and the macOS `Cmd+Option+I/J/C` equivalents on `keydown`;
- polls every second comparing outer and inner window dimensions with a 160 px threshold to guess whether DevTools is open;
- intercepts `contextmenu` inside the `<video>`;
- shows a copyright warning toast on any signal, throttled to once per 10 seconds via a `useRef` `timestamp`;
- falls back to “This lesson does not have a video yet” when `videoKey` is empty.

Video URLs are public bucket URLs, not signed — a determined user who reads the DOM can fetch the file.

PDF • `components/ui/SecurePDFViewer.tsx`

- Renders page by page onto a `<canvas>` with `pdfjs-dist`, so the browser’s native PDF UI — with its download and print buttons — never appears.
- Custom controls only: previous / next page, zoom, rotate, retry on error.
- Source URLs are 1-hour presigned GETs fetched on demand, so a copied URL expires.
- `pdfjs-dist` is imported **lazily** through a module-level memoized promise:


```

let pdfjsPromise: Promise<PdfJs> | null = null;
function loadPdfJs() {
  if (!pdfjsPromise) {
    pdfjsPromise = import("pdfjs-dist").then((lib) => {
      lib.GlobalWorkerOptions.workerSrc = "/pdf.worker.min.js";
      return lib;
    });
  }
  return pdfjsPromise;
}

```

A static import crashes server-side rendering because pdf.js touches `DOMMatrix` during module initialization, even inside a `"use client"` component.

A second, smaller copy lives at `app/(public)/courses/[slug]/_components/SecurePDFViewer.tsx` (181 lines vs 358) — see §24.

17 Abuse protection

`lib/arcjet.ts` exports a server-only base client keyed on a `"fingerprint"` characteristic with a single `shield({ mode: "LIVE" })` rule, and re-exports the rule builders so call sites import from one place.

SURFACE	RULES
POST /api/auth/[...all]	On /api/auth/sign-up with an email body: <code>protectSignup</code> — blocks <code>DISPOSABLE</code> / <code>INVALID</code> / <code>NO_MX_RECORDS</code> , disallows all bots, sliding window 5 per 2 min. Otherwise <code>detectBot</code> , plus <code>slidingWindow</code> on other sign-up shapes. Fingerprint: user id when signed in, else client IP via <code>@arcjet/ip</code> .
POST /api/s3/upload DELETE /api/s3/delete	<code>fixedWindow 5/min</code> , fingerprint = admin user id
CreateCourse, editCourse, deleteCourse, enrollInCourseAction	<code>fixedWindow 5/min</code> , fingerprint = user id, via <code>request()</code> from <code>@arcjet/next</code>

Denials become user-facing messages: rate limit → “You have been blocked due to rate limiting”; bot → “You are a bot! ...”. For auth specifically, per-reason email messages return `400`, rate limits `429`, and everything else `403`.

The auth route currently logs every Arcjet decision, and the S3 routes return the placeholder string `"dudde not good"` with a `429`.

18 UI system

AREA	IMPLEMENTATION
Tokens	<code>app/globals.css</code> defines the full shadcn token set in <code>oklch</code> for <code>:root</code> and <code>.dark</code> , plus font stacks, <code>--radius: 0.15rem</code> , a shadow scale, sidebar tokens and chart colors. Dark mode is a class variant — <code>@custom-variant dark (&:is(.dark *))</code> — driven by <code>next-themes</code> .
Primitives	<code>components/ui/</code> holds the shadcn/ui set (button, card, dialog, drawer, sheet, sidebar, table, tabs, select, form, input-otp, calendar, chart, sonner, tooltip...). <code>components.json</code> : <code>new-york</code> , <code>neutral</code> , CSS variables, lucide icons.
Shells	<code>/admin</code> and <code>/dashboard</code> both use <code>SidebarProvider</code> + <code>SidebarInset</code> + <code>SiteHeader</code> , with <code>--sidebar-width</code> and <code>--header-height</code> set inline on the provider.
Rich text	<code>Editor.tsx</code> (TipTap, StarterKit, TextAlign) and <code>Menubar.tsx</code> . Persisted as an HTML string and rendered by <code>RenderDescription.tsx</code> via <code>@tiptap/html</code> + <code>html-react-parser</code> .
Loading	Suspense with dedicated skeletons: <code>AdminCourseCardSkeleton</code> , <code>PublicCourseCardSkeleton</code> , <code>MaterialsSkeleton</code> , <code>LessonSkeleton</code> .
Empty states	<code>components/general/EmptyState.tsx</code> — <code>title</code> , <code>description</code> , <code>buttonText</code> , <code>href</code> .
Feedback	One <code><Toaster closeButton position="bottom-center" /></code> in the root layout; everything toasts through Sonner.
Responsive	<code>useIsMobile()</code> (768 px <code>matchMedia</code>) plus <code>lg</code> : branching. The course sidebar has separate desktop and mobile (<code>sheet</code>) implementations.
Typography	<code>Geist</code> and <code>Geist Mono</code> via <code>next/font/google</code> , exposed as <code>--font-geist-sans</code> / <code>--font-geist-mono</code> .

19 Shared hooks and helpers

EXPORT	FILE	NOTES
<code>tryCatch(promise)</code>	<code>hooks/try-catch.ts</code>	Discriminated <code>{ data, error: null } { data: null, error }</code> . The standard way action results are consumed.
<code>useCourseProgress</code>	<code>hooks/use-course-progress.ts</code>	Memoized totals and percentage
<code>useConfetti()</code>	<code>hooks/use-confetti.ts</code>	<code>triggerConfetti()</code> , five-burst preset

EXPORT	FILE	NOTES
<code>useSignOut()</code>	<code>hooks/use-signout.ts</code>	Signs out, redirects to <code>/</code> , toasts. Filename typo noted in §24.
<code>useIsMobile()</code>	<code>hooks/use-mobile.ts</code>	<code>matchMedia</code> at 768 px
<code>useConstructUrl(key)</code>	<code>hooks/use-construct-url.ts</code>	Not a hook — a pure function. Returns absolute URLs unchanged, otherwise prefixes the bucket host.
<code>cn(...)</code>	<code>lib/utls.ts</code>	<code>clsx</code> + <code>tailwind-merge</code>
<code>ApiResponse</code>	<code>lib/types.ts</code>	<code>{ status, message }</code>
<code>prisma</code>	<code>lib/db.ts</code>	Global singleton outside production so hot reload does not exhaust connections

20 Validation schemas

`lib/zodSchemas.ts` is the single source of form validation. Its constant arrays double as the option lists in admin `Select` s, so a new level or category is a one-line change.

```

courseLevels      = ["Beginner", "Intermediate", "Advanced"]
courseStatus      = ["Draft", "Published", "Archived"]
universities      = ["UJ", "PETRA"]
courseCategories = ["Development", "Business", "Finance", "IT & Software",
                    "Office Productivity", "Personal Development", "Design",
                    "Marketing", "Health & Fitness", "Music", "Teaching & Academics"]

```

SCHEMA	FIELDS
<code>courseSchema</code>	<code>title</code> (3–100), <code>description</code> (≥3), <code>fileKey</code> (≥1), <code>price</code> (coerced, ≥1), <code>duration</code> (coerced, 1–500), <code>level</code> , <code>category</code> , <code>smallDescription</code> (3–200), <code>slug</code> (≥3), <code>status</code> , <code>instructorId</code> (uuid, optional), <code>university</code>
<code>chapterSchema</code>	<code>name</code> (≥3), <code>courseId</code> (uuid)
<code>lessonSchema</code>	<code>name</code> (≥3), <code>chapterId</code> (uuid), <code>courseId</code> (uuid), <code>description</code> (≥3, optional), <code>videoKey</code> (optional), <code>thumbnailKey</code> (optional)

Exported inference types: `CourseSchemaType`, `ChapterSchemaType`, `LessonSchemaType`. `slugify` generates the slug from the title in `CourseCreationForm`.

KEEP IN SYNC These enum arrays are duplicated from the Prisma enums, not generated. Every server action re-validates with `safeParse` — client validation is a UX affordance, not a trust boundary.

21 Local development

```
pnpm install      # runs prisma generate via postinstall
pnpm dev          # next dev --turbopack
pnpm build        # next build
pnpm start        # next start
pnpm lint         # next lint
```

DATABASE

```
npx prisma db push # apply schema.prisma (there are no migrations)
npx prisma studio  # inspect data
```

`.env` in the repo holds **placeholder** values for local work only; real credentials belong in the hosting provider's dashboard. With placeholders:

- GitHub sign-in does not work.
- Email OTP does not deliver — but the code is printed to the dev server console by `lib/auth.ts`, which is the intended local sign-in path.
- S3 uploads, video playback and PDF viewing do not work.
- Stripe checkout does not work.

Granting admin locally. Sign in once, then set `role = 'admin'` on your `user` row in Prisma Studio or SQL.

STRIPE WEBHOOKS

```
stripe listen --forward-to localhost:3000/api/webhook/stripe
```

Put the printed signing secret in `STRIPE_WEBHOOK_SECRET`.

22 Build and deployment

Target platform is Vercel. The checklist below must hold for a working production deployment.

- **Prisma client.** `@prisma/nextjs-monorepo-workaround-plugin` is a dependency and `prisma generate` runs on `postinstall`, so the client exists at build time.
- **Environment.** `lib/env.ts` validates every variable at import; a misconfigured deployment fails the build. Set all variables from §4 in the hosting dashboard.
- **Image host.** `next.config.ts` hardcodes `lms-alamzoh.fly.storage.tigris.dev`. Pointing at a different bucket requires editing this file as well as `NEXT_PUBLIC_S3_BUCKET_NAME_IMAGES`.
- **Stripe webhook.** Register `https://<host>/api/webhook/stripe` for `checkout.session.completed` and set `STRIPE_WEBHOOK_SECRET` to its signing secret.
- **Auth URL.** `BETTER_AUTH_URL` must be the public origin — it drives OAuth callbacks *and* Stripe return URLs.

- **Email.** Resend needs a verified sending domain. The `from` address is the shared sandbox `onboarding@resend.dev`, hardcoded in `lib/auth.ts`, which will not deliver reliably in production.

23 Maintenance scripts

Run with `tsx`; they are not wired into `package.json`.

SCRIPT	PURPOSE
<code>scripts/cleanup-orphans.ts</code>	Deletes <code>Chapter</code> rows whose <code>courseId</code> no longer exists
<code>scripts/cleanup-enrollments.ts</code>	Deletes <code>Enrollment</code> rows whose <code>userId</code> or <code>courseId</code> no longer exists

```
npx tsx scripts/cleanup-orphans.ts
npx tsx scripts/cleanup-enrollments.ts
```

Both use `notIn` against a full list of valid ids — fine at current volumes, but it will not scale. Their existence points to historical data created before cascade rules were in place.

24 Known issues and technical debt

Ordered roughly by impact. Each item is a factual observation about the current code.

Correctness and security HIGH IMPACT

01 No migration history

`prisma/migrations/` does not exist; the schema is applied with `db push`. Changes are not reproducible or reviewable, and production drift is undetectable.

02 `editCourse` is scoped to the creator

The update's `where` includes `userId: user.user.id`, so admin A cannot edit a course created by admin B — the update silently affects zero rows and the action reports failure.

03 Stripe prices go stale

`CreateCourse` creates the price once; `editCourse` changes `Course.price` without touching Stripe. The displayed price diverges from the charged price.

04 Webhook throws instead of responding

Missing `metadata.courseId` or an unknown `stripeCustomerId` throws, producing a 500 and an indefinite retry loop. The handler also trusts `metadata.enrollmentId` without checking it belongs to the resolved user.

05 Materials upload runs through the server

The whole file is buffered in memory, bounded by the ~4.5 MB Vercel body limit. Videos already use presigned PUTs; materials should too.

06 Video URLs are unsigned

Lesson video and thumbnail URLs are public. The DevTools and right-click deterrents don't stop anyone reading the DOM. Real protection needs signed URLs or token-gated streaming.

07 Deleting a course leaves storage and Stripe behind

`deleteCourse` removes rows but never deletes S3 objects or archives the Stripe product, so both accumulate.

08 `/api/s3/upload` does not restrict type or size

`contentType` and `size` are required but not validated, and `isImage` is parsed then ignored. Any admin can presign any upload.

09 Role checks are string comparisons

`role === "admin"` is spread across the codebase with no shared constant, and `"instructor"` grants nothing — the role is purely presentational today.

Consistency and duplication MEDIUM

10 Two `SecurePDFViewer` components

`components/ui/` (358 lines) and `app/(public)/courses/[slug]/_components/` (181 lines). They will drift.

11 **Two `getInstructors()` functions**

In `courses/create/actions.ts` and `instructors/actions.ts`, returning different shapes.

12 **Two `CoursesFilters` components**

Near-identical logic for the public catalog and admin.

13 **Hand-rolled authorization in `/api/courses/[courseId]/materials`**

Duplicates `requireCourseAccess` from `lib/api-auth.ts` inline.

14 **Enum values are duplicated**

Between `prisma/schema.prisma` and `lib/zodSchemas.ts`, with no generation step.

15 **University is an enum, not a table**

A third university needs a schema change, a Zod change, and edits to `getUniversityStats()`'s two hardcoded branches.

16 **Instructor university is inferred**

Taken from their first course, defaulting to `"UJ"`. Instructors at both universities, or none, are misreported.

Performance SCALING

17 **Student statistics computed in application code**

`getStudentsStats()` and `getStudents()` load every enrollment, chapter, lesson id and progress row, then loop — $O(\text{lessons} \times \text{students})$ per page view. Should be aggregate SQL.

18 **`/courses` is force-dynamic and unfiltered server-side**

All published courses are serialized on every request and filtered in the browser.

19 **No explicit database indexes**

No `@index` declarations; lookups on `Course.slug`, `Course.status` and `Material.courseId` rely on implicit unique and foreign-key indexes.

Cosmetic LOW

20 **Dead chart data**

`dummyEnrollmentData` in `chart-area-interactive.tsx`, kept alive by a file-level `eslint-disable`.

21 **Template scaffolding**

`DashboardAppSidebar` carries unused `navClouds` / `documents`; admin “Settings” and “Get Help” link to `#`.

22 **Placeholder error strings reach users**

`"dudde not good"` from the S3 routes on 429.

23 **Typos in copy and filenames**

`use-singout.ts`, “Singed out Successfully”, “created succesfully”, “alredy enrolled”.

24 **Noisy auth logging**

`app/api/auth/[...all]/route.ts` logs every Arcjet decision on every request.

25 Swallowed errors

Several admin data functions return `[]` or `null` on failure (`admin-get-materials.ts`), so a database outage renders as an empty list.

25 Conventions

Patterns to follow when extending the codebase.

`import "server-only"` everywhere sensitive

Every module in `app/data/` and the sensitive `lib/` files (`arcjet` , `stripe` , `S3Client` , `api-auth`). A client-side import becomes a build error.

Guard first, then `try/catch`

Authorization runs before the `try` block so a 401 / 403 cannot be swallowed into a 500.

Redirect guards for pages, JSON guards for routes

`requireAdmin` / `requireUser` for navigations; `requireApiAdmin` / `requireApiUser` / `requireCourseAccess` for `fetch` callers.

`cache()` around session guards

A request resolves the session at most once.

Return `ApiResponse` from every action

Consume it through `tryCatch` and a Sonner toast.

Validate with Zod inside the action

Never trust the client form's validation.

Derive types from queries

Use `Awaited<ReturnType<typeof fn>>` rather than hand-written DTOs, so widening a `select` widens the component contract.

`revalidatePath` after every mutation

Whenever a server-rendered page displays the changed data.

Colocate

Route-specific components in `_components/` , route-specific actions in `actions.ts` .

`position` is 1-based and renumbered on delete

Inside a transaction, so gaps never appear.

Object keys may be absolute URLs

Anything turning a `fileKey` into a URL must handle `/^https?:\\/\\/i` — see `useConstructUrl` and `/api/materials/[id]/view` .

END OF DOCUMENT